# OS ORANGE ASSIGNMENT

## Name: Arnav Sharath Hanchanur

## SRN: PES2UG24CS084

## Github link:

## Screenshots:

Phase 1

1 A)

```
arnav@arnav-VirtualBox:~/PES2UG24CS084-pes-vcs$ make test_objects
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -o test_objects test_objects.o object.o -lcrypto
arnav@arnav-VirtualBox:~/PES2UG24CS084-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b
9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe
9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
arnav@arnav-VirtualBox:~/PES2UG24CS084-pes-vcs$
```

1 B)

```
arnav@arnav-VirtualBox:~/PES2UG24CS084-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
arnav@arnav-VirtualBox:~/PES2UG24CS084-pes-vcs$
```

Phase 2:

2 A) Output of ./test_tree:

```
arnav@arnav-VirtualBox:~/PES2UG24CS084-pes-vcs$ make test_tree
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
arnav@arnav-VirtualBox:~/PES2UG24CS084-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
arnav@arnav-VirtualBox:~/PES2UG24CS084-pes-vcs$
```

2 B)

```
arnav@arnav-VirtualBox:~/PES2UG24CS084-pes-vcs$ xxd .pes/objects/2a/ae6c35c94fcf
b415dbe95f408b9ce91ee846ed
00000000: 6865 6c6c 6f20 776f 726c 640a            hello world.
arnav@arnav-VirtualBox:~/PES2UG24CS084-pes-vcs$
```

Phase 3

3 A)

```
arnav@arnav-VirtualBox:~/PES2UG24CS084-pes-vcs$ make pes
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
arnav@arnav-VirtualBox:~/PES2UG24CS084-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
arnav@arnav-VirtualBox:~/PES2UG24CS084-pes-vcs$ echo "hello" > file1.txt
arnav@arnav-VirtualBox:~/PES2UG24CS084-pes-vcs$ echo "world" > file2.txt
arnav@arnav-VirtualBox:~/PES2UG24CS084-pes-vcs$ ./pes add file1.txt file2.txt
arnav@arnav-VirtualBox:~/PES2UG24CS084-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  Makefile
  untracked:  test_sequence.sh
  untracked:  commit.c
  untracked:  object.c
  untracked:  test_objects
  untracked:  test_tree
  untracked:  tmp.txt
  untracked:  pes.c
  untracked:  .gitignore
  untracked:  pes.h
  untracked:  commit.h
  untracked:  test_objects.c
  untracked:  index.c
  untracked:  tree.c
  untracked:  test_tree.c
  untracked:  index.h
  untracked:  README.md
  untracked:  tree.h

arnav@arnav-VirtualBox:~/PES2UG24CS084-pes-vcs$
```

3 B)

```
arnav@arnav-VirtualBox:~/PES2UG24CS084-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 17764
48795 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 17764
48803 6 file2.txt
arnav@arnav-VirtualBox:~/PES2UG24CS084-pes-vcs$
```

Phase 4:

4 A)

```
arnav@arnav-VirtualBox:~/PES2UG24CS084-pes-vcs$ ./pes log
commit b6c8719e9d83e4adaa57f934b822f06eab9bf018d64d424707c7e687b1851871
Author: PES User <pes@localhost>
Date:   1776450678

    Add farewell

commit 3742d7b3c662119bb33ead8ba84510389550facfe3af8c2706681d440060620d
Author: PES User <pes@localhost>
Date:   1776450657

    Add world

commit 2e29b63fa2e3b2b2ee341fe4f828f097ee134d06923cc2edcd1f0c66062437cc
Author: PES User <pes@localhost>
Date:   1776450636

    Initial commit

arnav@arnav-VirtualBox:~/PES2UG24CS084-pes-vcs$
```

4 B)

```
arnav@arnav-VirtualBox:~/PES2UG24CS084-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/2a/ae6c35c94fcfb415dbe95f408b9ce91ee846ed
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/2e/29b63fa2e3b2b2ee341fe4f828f097ee134d06923cc2edcd1f0c66062437cc
.pes/objects/37/42d7b3c662119bb33ead8ba84510389550facfe3af8c2706681d440060620d
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/96/c0d5c28dd3b719606c36723cd16aefbc73b3dc8b74de61070183357d6968b9
.pes/objects/b6/c8719e9d83e4adaa57f934b822f06eab9bf018d64d424707c7e687b1851871
.pes/objects/c9/6f2fc25394979be592d5cf80c5d78ec792ff13dde17f838449fbcdfbb7df0d
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
arnav@arnav-VirtualBox:~/PES2UG24CS084-pes-vcs$
```

4 C)

```
arnav@arnav-VirtualBox:~/PES2UG24CS084-pes-vcs$ cat .pes/refs/heads/main
b6c8719e9d83e4adaa57f934b822f06eab9bf018d64d424707c7e687b1851871
arnav@arnav-VirtualBox:~/PES2UG24CS084-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
arnav@arnav-VirtualBox:~/PES2UG24CS084-pes-vcs$
```

Integration-test:

```
arnav@arnav-VirtualBox:~/PES2UG24CS084-pes-vcs$ make test-integration
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 52ac502fc968... Initial commit

Log after first commit:
commit 52ac502fc9686ebfd06e64435702e472df86588277fbaa773857980ecd5b5ed9
Author: PES User <pes@localhost>
Date:   1776450830

    Initial commit


--- Second Commit ---
Committed: 698ea7dea9b1... Update file.txt

--- Third Commit ---
Committed: f56c958fbb43... Add farewell

--- Full History ---
commit f56c958fbb4363c8b70aaf95b0a3dd3130ebf0e53a47e626d1bfe3d261c9772e
Author: PES User <pes@localhost>
Date:   1776450830

    Add farewell
```

```
commit 698ea7dea9b1c10cd9d2678cbc360fcd9039cf005fa66c84de14cddb73c9f65e
Author: PES User <pes@localhost>
Date:   1776450830

    Update file.txt

commit 52ac502fc9686ebfd06e64435702e472df86588277fbaa773857980ecd5b5ed9
Author: PES User <pes@localhost>
Date:   1776450830

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
f56c958fbb4363c8b70aaf95b0a3dd3130ebf0e53a47e626d1bfe3d261c9772e

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/52/ac502fc9686ebfd06e64435702e472df86588277fbaa773857980ecd5b5ed9
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/69/8ea7dea9b1c10cd9d2678cbc360fcd9039cf005fa66c84de14cddb73c9f65e
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
.pes/objects/f5/6c958fbb4363c8b70aaf95b0a3dd3130ebf0e53a47e626d1bfe3d261c9772e

=== All integration tests completed ===
arnav@arnav-VirtualBox:~/PES2UG24CS084-pes-vcs$
```

Phase 5 and 6:

Q 5.1:

Update the .pes pointers by first checking if the branch file exists in .pes/refs/heads/. Then read the commit hash from that file and update .pes/HEAD to point to it using a safe temporary file and rename operation.

Next, reconstruct the project state by reading the commit object, getting its root tree, and recursively expanding all tree objects to map file paths to their corresponding blobs.

After that, update the working directory:

- Write each blob's content into its respective file with correct permissions
- Remove files that are no longer tracked

- Create directories if needed

Finally, rebuild the index so it matches the target tree, including updated metadata.

This process is complex because it must carefully handle file deletions, nested folders, permissions, and avoid losing any uncommitted changes.

Q 5.2:

For each tracked file, perform a 3-way comparison:

- Base → current HEAD version
- Staged → version in .pes/index
- Working → current file in the directory

Compare the current commit state with the target commit to see what has changed.

Before switching (checkout):

- If a file has local changes and the target also modifies it → stop and refuse checkout
- If an untracked file would be overwritten → also refuse checkout

This ensures that checkout does not overwrite or destroy any local, uncommitted work.

Q 5.3:

In a detached HEAD state, .pes/HEAD stores a commit hash directly instead of pointing to a branch. You can still make new commits, and each new commit points to the previous one, but no branch is tracking them.

The problem is that these commits are not referenced by any branch, so they can become unreachable.

To recover, you should create a branch pointing to these commits or switch to a branch after saving them. If you don't, garbage collection (GC) may eventually delete them.

Q 6.1:

Using a mark-and-sweep method:

- Mark phase: Start from all branch tips and traverse commits, trees, and blobs (using DFS or BFS), marking all reachable objects.
- Sweep phase: Go through all objects in .pes/objects and delete those that were not marked. Optionally remove empty folders.

A hash set is used to quickly check if an object has already been visited. The overall work depends on the number of reachable objects and total stored objects.

Q 6.2:

A race condition can happen during commits:

- Objects (blobs, trees, commit) are written first
- Then the branch reference is updated

If garbage collection runs before the reference is updated, it may think the new objects are unused and delete them. This can break the commit history.

To prevent this, real systems use:

- Locks and synchronization
- Reflogs and grace periods
- Delayed deletion of recent objects

This ensures objects are safely stored before references are updated.